

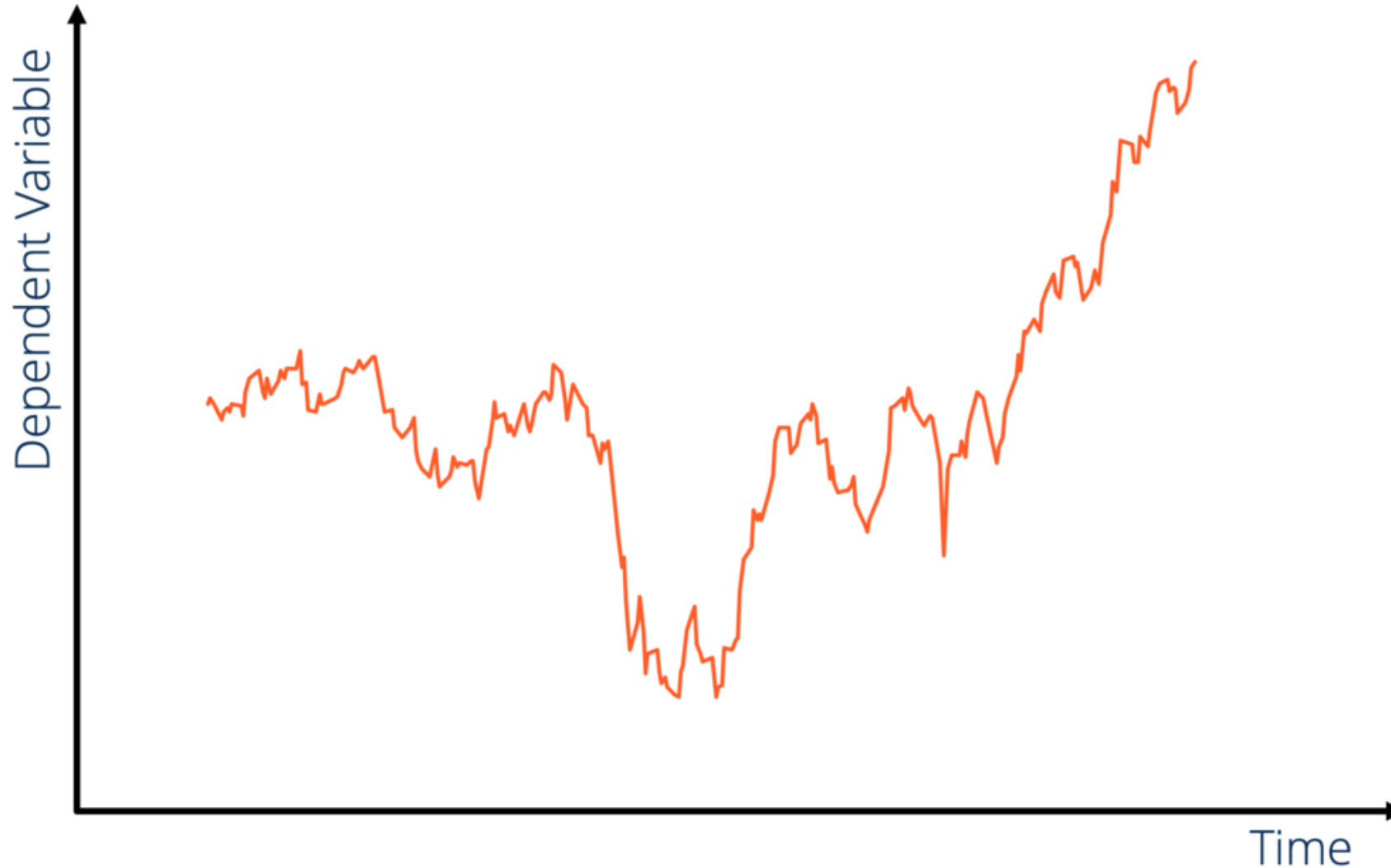
Time Series Analysis

Havana Rika

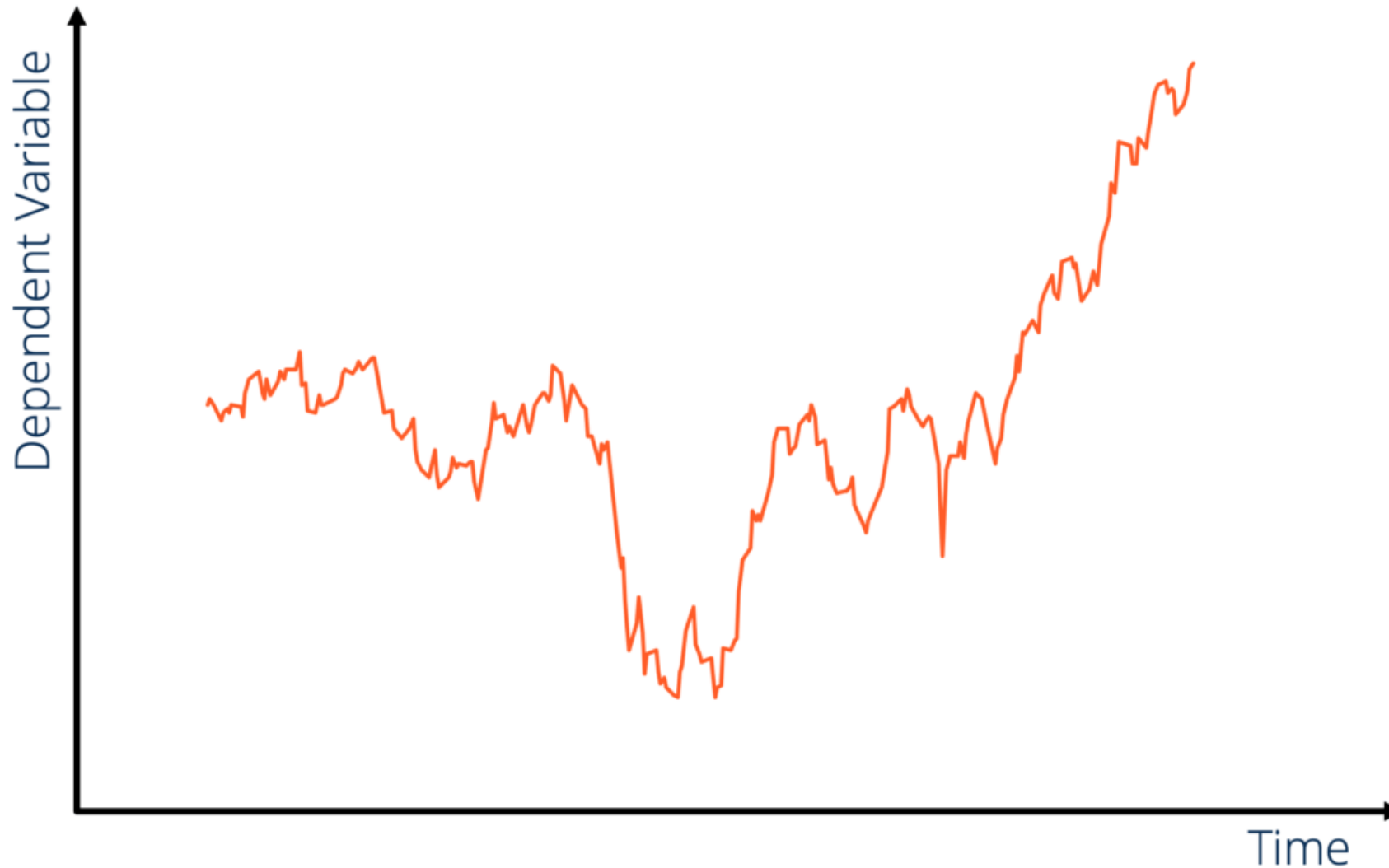


Ben-Gurion University
of the Negev

What is a time series?



What is a time series?

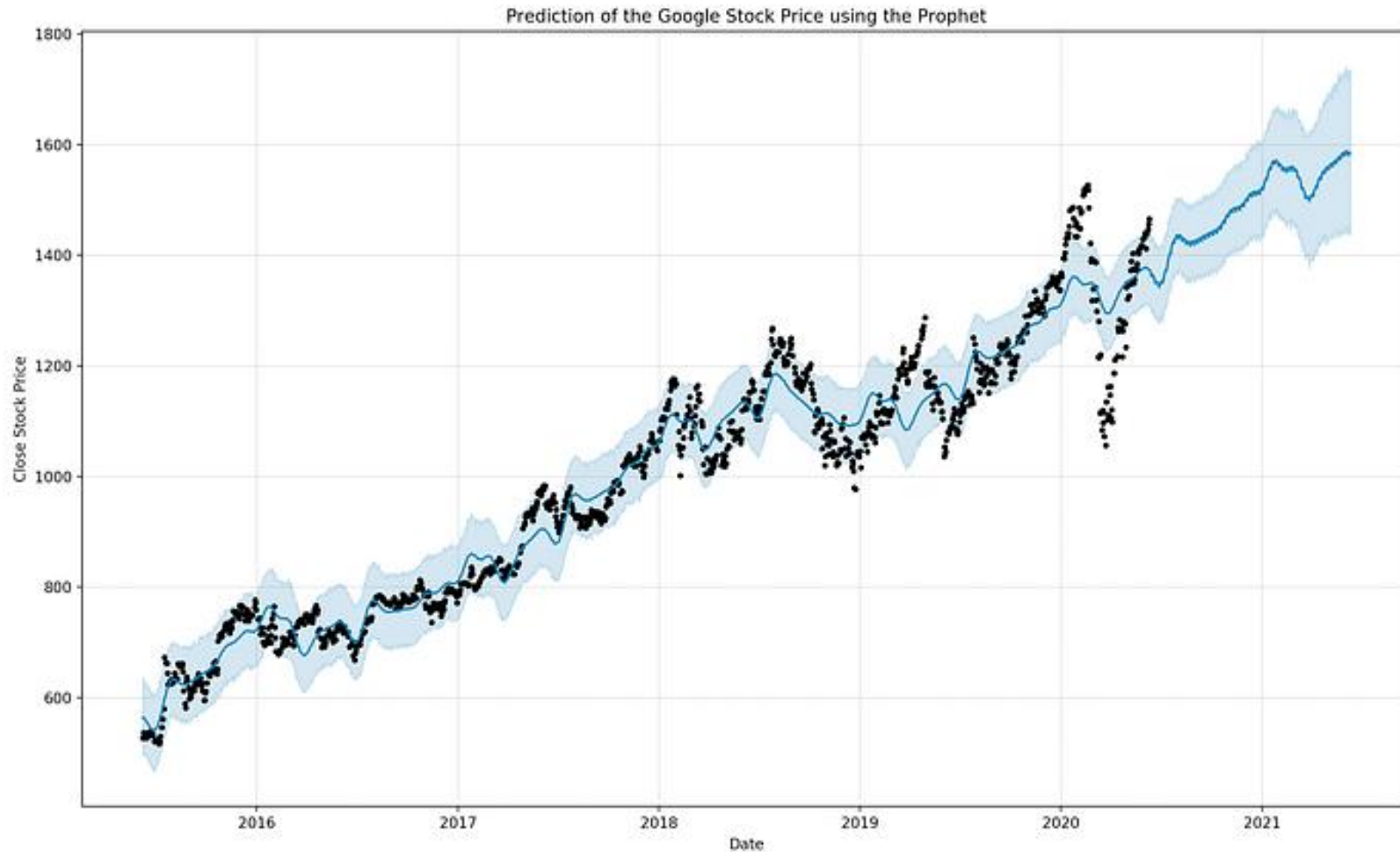




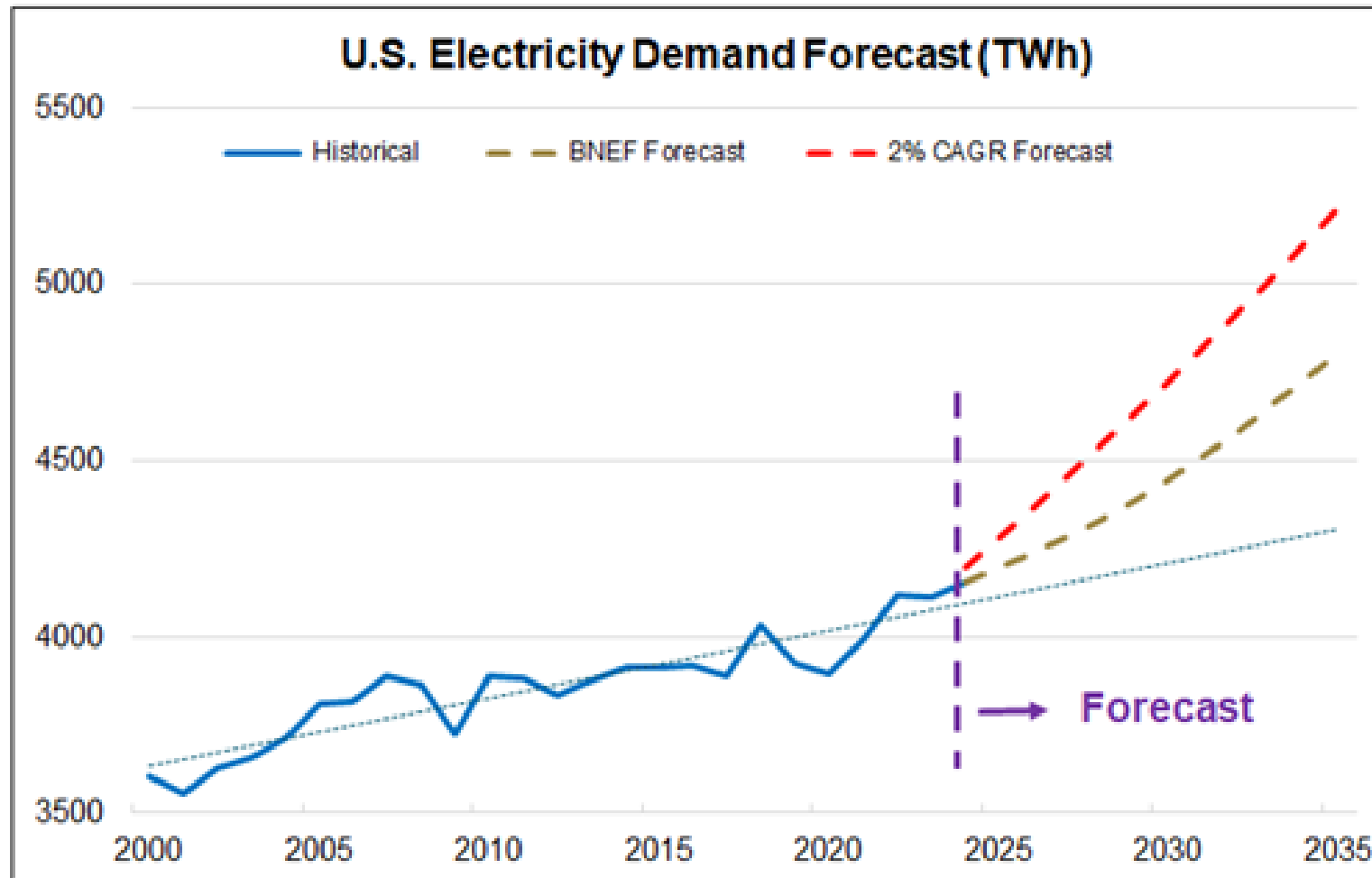
Time Series

A sequence of data points that occur in successive order over some period of time.

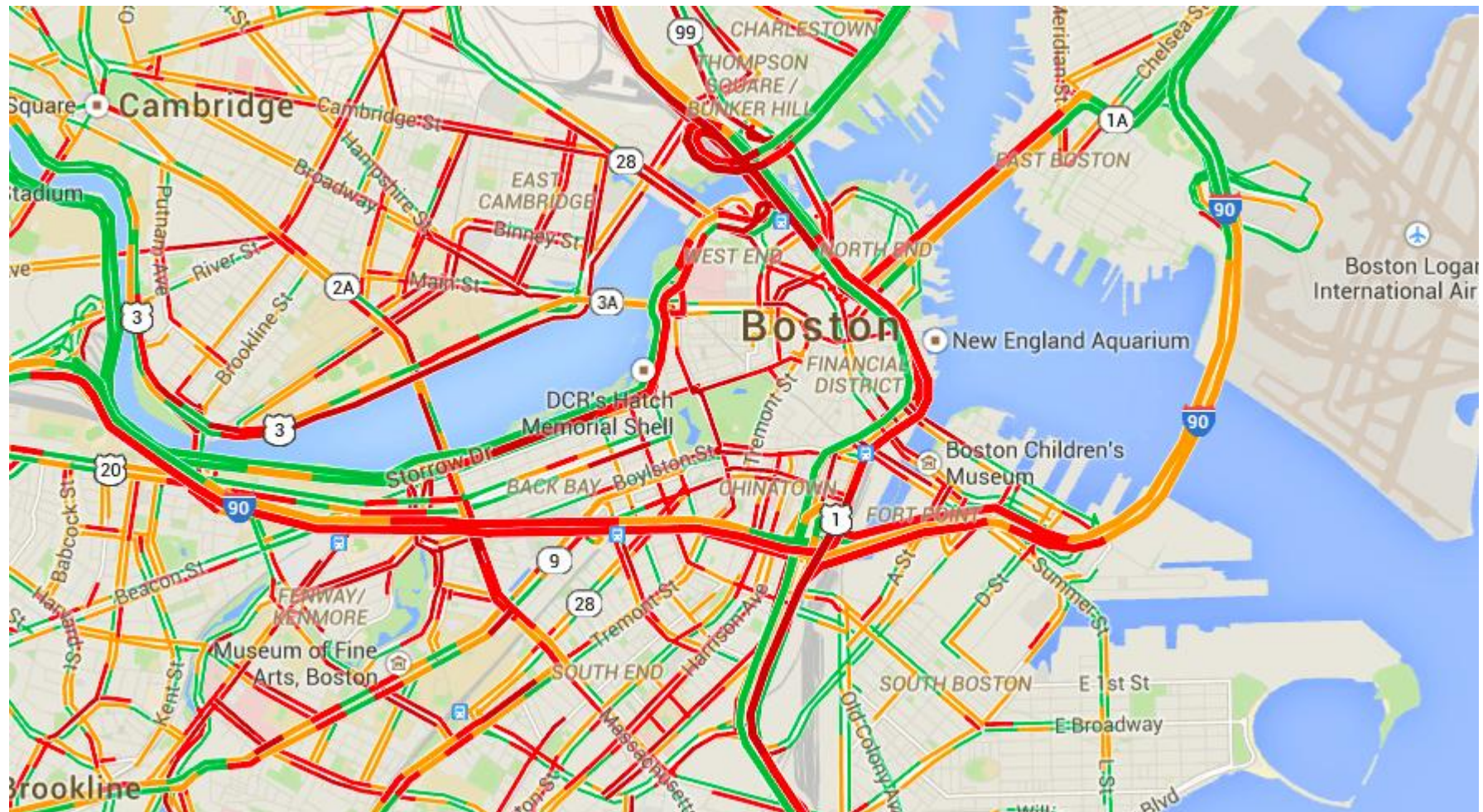
Stock price forecasting



Electricity demand prediction



Traffic flow prediction

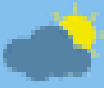
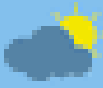


Weather forecasting

**Wave Height
Forecast**

**Earthquake
Forecast**

**Flood
Forecast**

SUN	MON	TUE	WED	THU	FRI	SAT
						
°C	°C	°C	°C	°C	°C	°C
°F	°F	°F	°F	°F	°F	°F
▲ 25	▲ 27	▲ 24	▲ 21	▲ 22	▲ 25	▲ 23
▼ 19	▼ 20	▼ 16	▼ 15	▼ 16	▼ 20	▼ 18

Sales forecasting



Disease outbreak prediction

COVID-19 Surveillance Data in the United States

Early Indicators

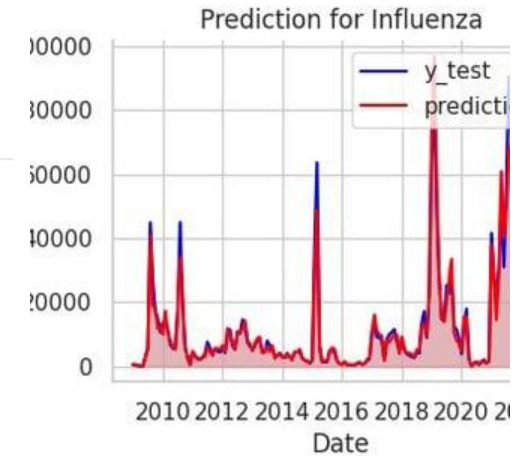
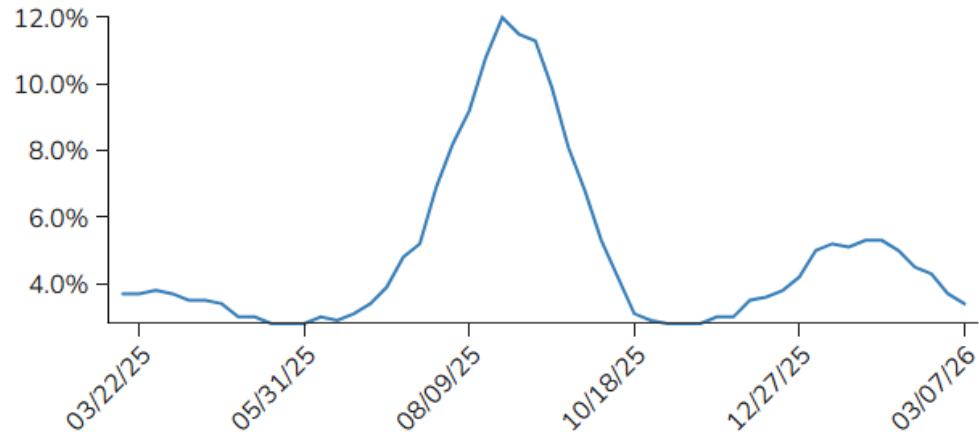
Test positivity (the percentage of total reported tests that are positive) and the percentage of total emergency department visits due to COVID-19 are key metrics to assess the impact of COVID-19 on communities. For public health professionals, these metrics act as early indicators of potential increases in COVID-19 activity.

% Test Positivity

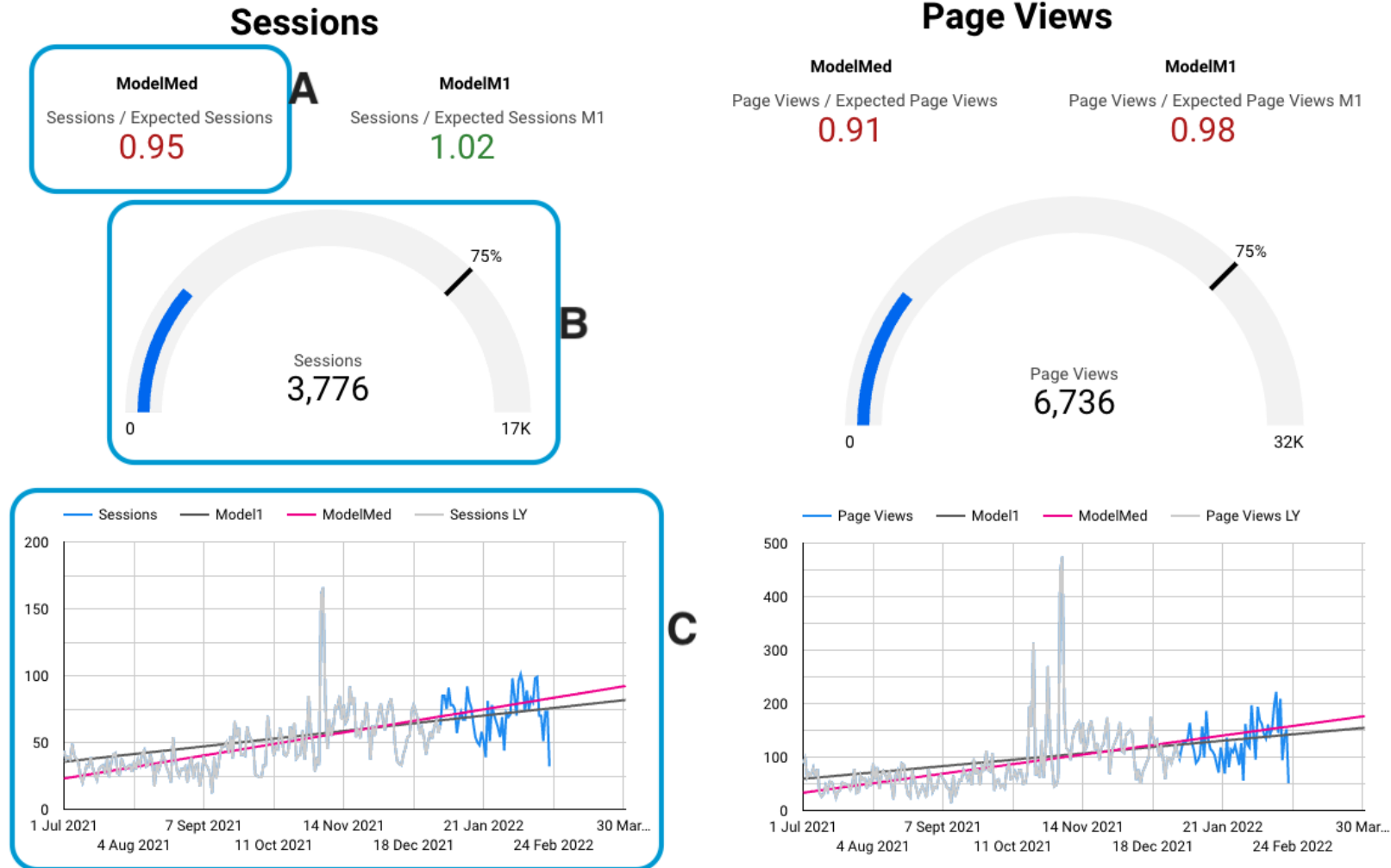
3.4%

Week ending 2026-03-07

Previous Week 3.7%



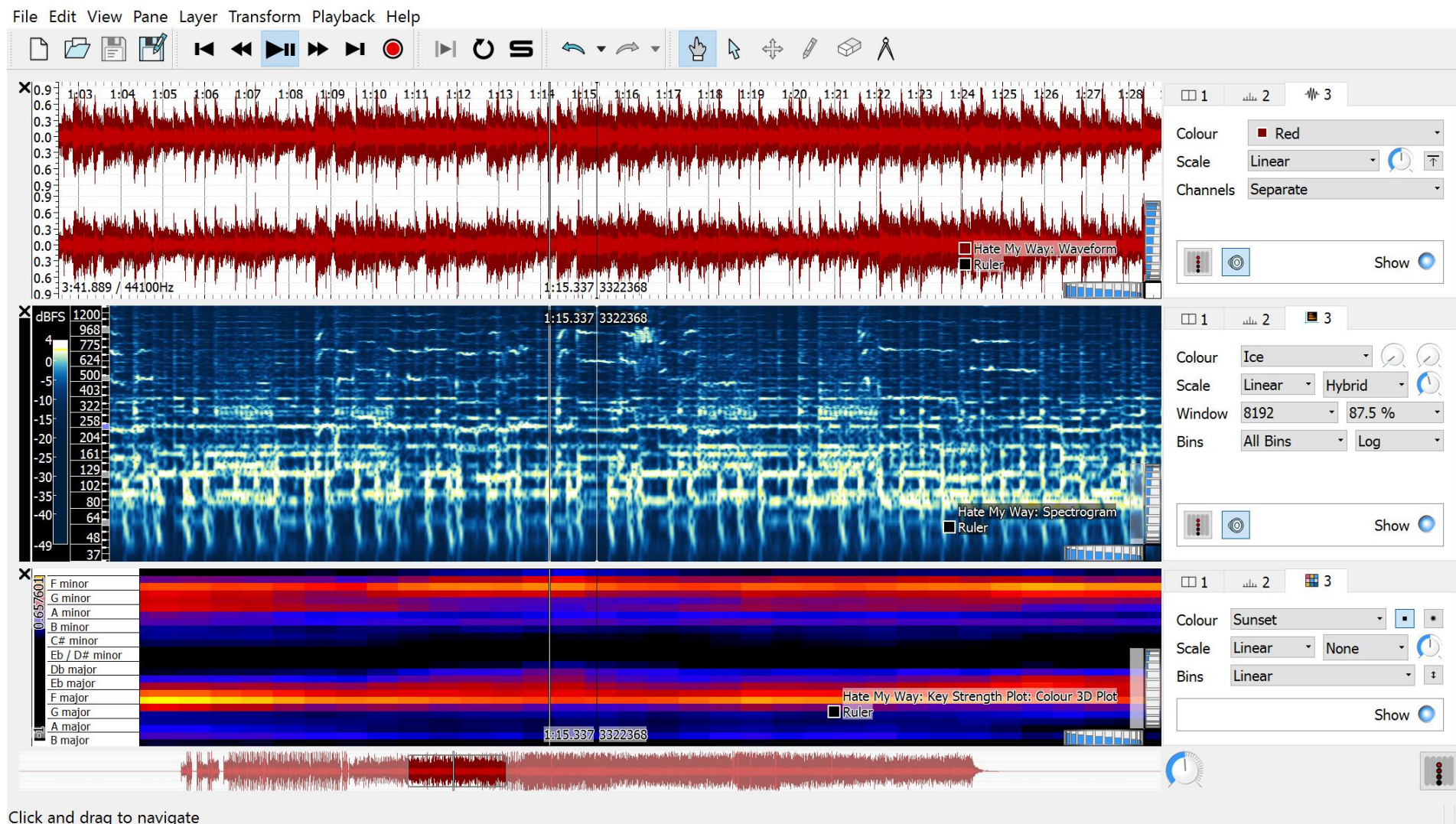
Website traffic prediction



Predictive maintenance for machines



Audio and speech signal prediction



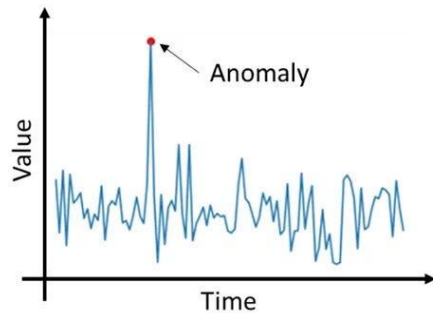
Player behavior prediction in games

iGaming CRM models powered by predictive analytics are redefining player retention, engagement, and loyalty.



Additional Problems

Anomaly detection

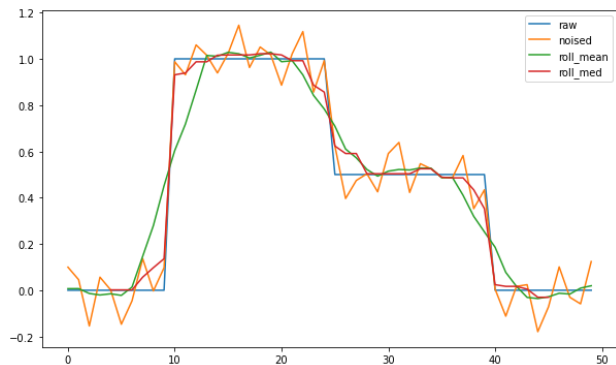


Causal analysis

Early warning systems

Similarity search

Denoising

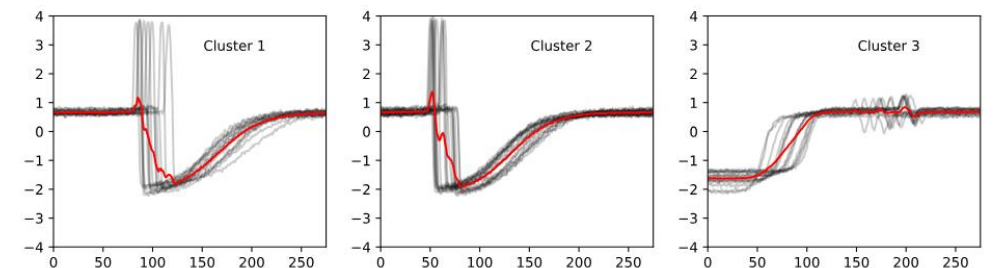


Trend/Seasonality detection



trendet - is a Python package for trend detection on stock time series data

Clustering of time series



Types of time series

- **Regular time series:**

This is the most common type of time series, where we have observations coming in at **regular intervals** of time, such as every hour or every month. For example, if we take a time series of **temperature in a city**, we will get the time series in a regular interval (whichever frequency we choose for observation).

- **Irregular time series:**

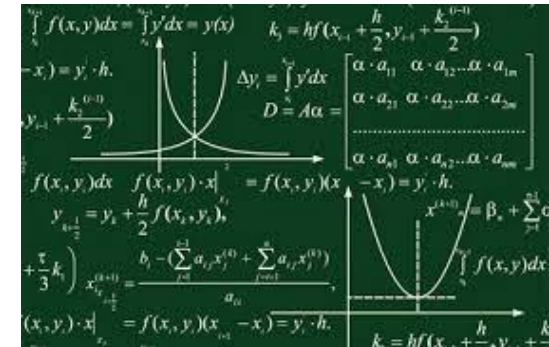
There are a few time series where we do not have observations at regular intervals of time. For example, consider we have a sequence of readings from **lab tests** of a patient. We see an observation in the time series only when the patient heads to the clinic and carries out the lab test, and this may not happen at regular intervals.

Data Generating Process (DGP)

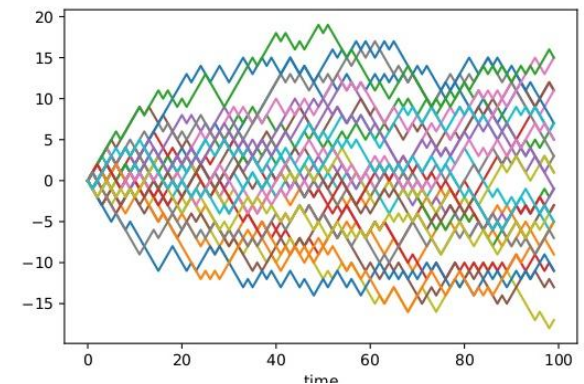
Time series data is produced by **stochastic** and **deterministic** processes.

The **deterministic** processes involve **quantities** that **evolve** in a **predictable manner over time**. An example of this is the radioactive decay of an element, where the remaining quantity diminishes according to a precise mathematical formula, leading to a consistent reduction over time.

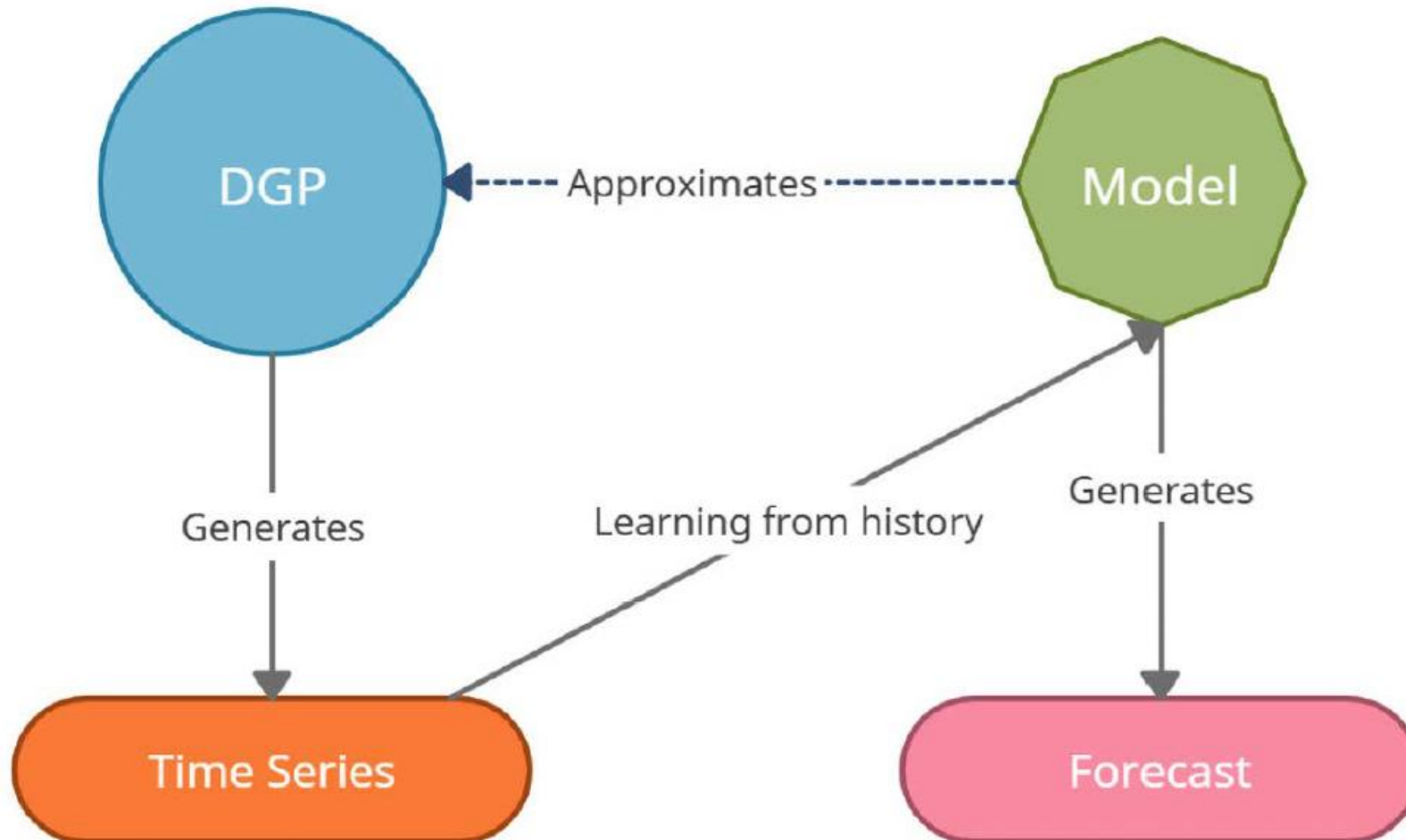
But most of the **interesting** time series (from a forecasting perspective) are generated by a **stochastic** process. A stochastic process is a way to describe how things change over time in a **random but somewhat predictable** manner, like how the weather changes daily with some patterns and probabilities involved.



The image is a collage of mathematical content. It includes the integral formula $\int_a^b f(x,y)dx = \int_a^b y'dx = y(x)$, the Runge-Kutta coefficients $k_1 = hf(x_i, y_i)$, $k_2 = hf(x_i + \frac{h}{2}, y_i + \frac{k_1}{2})$, and $k_3 = hf(x_i + h, y_i + k_1)$. It also shows the Taylor series expansion $y(x_i + h) = y(x_i) + h y'(x_i) + \frac{h^2}{2} y''(x_i) + \frac{h^3}{6} y'''(x_i) + \dots$. A matrix $D = A\alpha$ is shown with $\alpha = [\alpha_1, \alpha_2, \dots, \alpha_m]^T$ and $A = [a_{11}, a_{12}, \dots, a_{1m}; a_{21}, a_{22}, \dots, a_{2m}; \dots; a_{n1}, a_{n2}, \dots, a_{nm}]$. A graph shows a curve $y(x)$ and its approximation $y(x_i + h)$. Another graph shows a curve $y(x)$ and its approximation $y(x_i + h)$ with a vertical dashed line at $x_i + h$. The formula $k_1 = hf(x_i, y_i)$ is repeated at the bottom.



Data Generating Process (DGP)



Generating Synthetic Time Series

Synthetic time series are an excellent tool to **understand** the time series space, **experiment** with different techniques, and even **test** new models or modeling setups.

These time series are designed to be **predictable**, even though a **bit challenging**.

Let's **generate** a few time series using a set of **fundamental building blocks**.

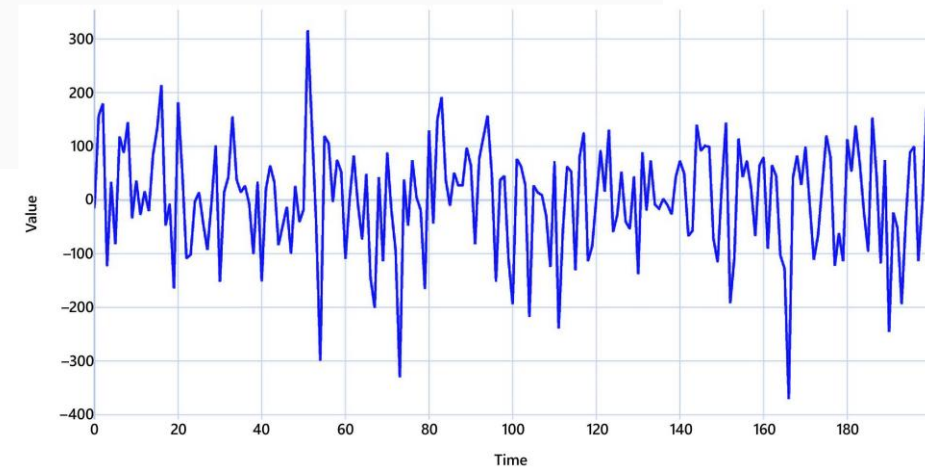
You can **mix & match** any of these components to generate a time series of arbitrary complexity.

White and Red Noise

An **extreme case of a stochastic process** that generates a time series is a white noise process. It has a sequence **of random numbers with zero mean and constant variance**.

This is also one of the most **popular** assumptions of **noise** in a time series.

```
# Generate the time axis with sequential numbers upto 200  
time = np.arange(200)  
# Sample 200 hundred random values  
values = np.random.randn(200)*100  
plot_time_series(time, values, "White Noise")
```



Red Noise

Red noise, on the other hand, has **zero mean** and **constant variance** but is **serially correlated in time**.

$$x_{j+1} = r \cdot x_j + (1 - r^2)^{\frac{1}{2}} \cdot w$$

This serial correlation or redness is parameterized by a **correlation coefficient r** , where **w** is a random sample from a **white noise** distribution.

Red Noise

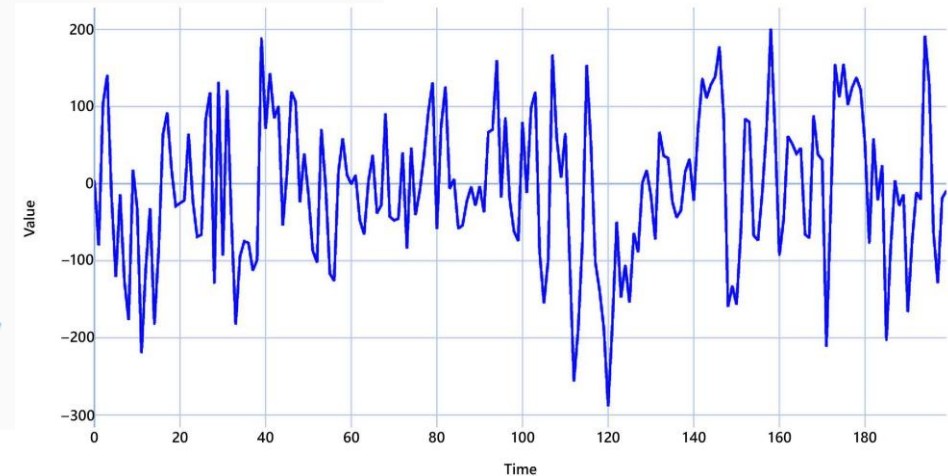
```
# Setting the correlation coefficient
r = 0.4

# Generate the time axis
time = np.arange(200)

# Generate white noise
white_noise = np.random.randn(200)*100

# Create Red Noise by introducing correlation between subsequent values in the
white noise

values = np.zeros(200)
for i, v in enumerate(white_noise):
    if i==0:
        values[i] = v
    else:
        values[i] = r*values[i-1]+ np.sqrt((1-np.power(r,2))) *v
plot_time_series(time, values, "Red Noise Process")
```



Cyclical or Seasonal Signals

Among the **most common signals** you see in time series are **seasonal** or **cyclical** signals.

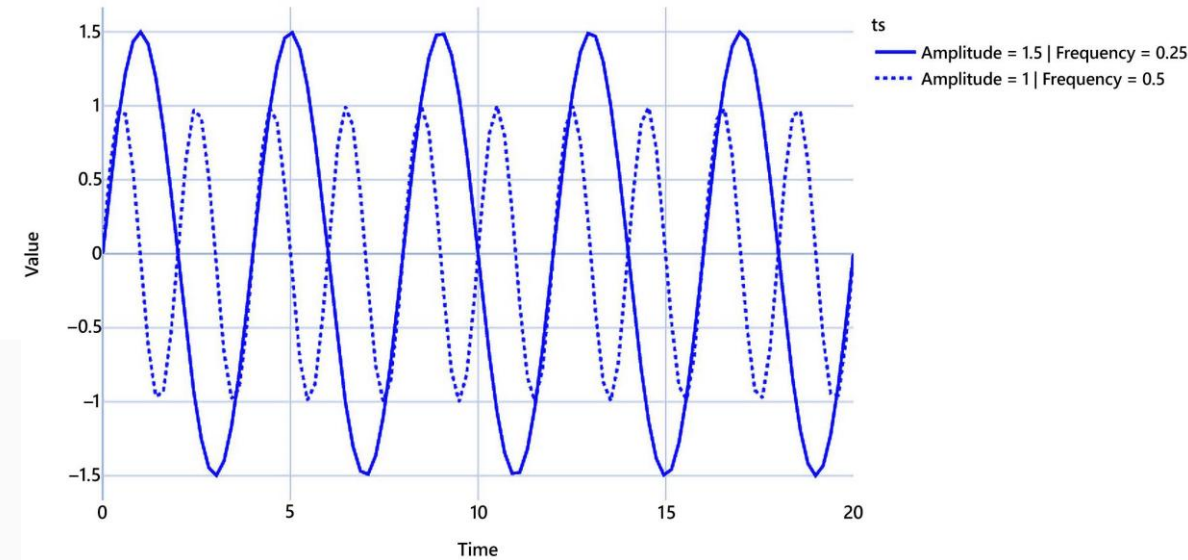
A very useful library to generate the rest of the time series is **TimeSynth**. It has all kinds of DGPs that you can mix and match and create an authentic synthetic time series.



<https://github.com/TimeSynth/TimeSynth>

Cyclical or Seasonal Signals

```
#Sinusoidal Signal with Amplitude=1.5 & Frequency=0.25
signal_1 = ts.signals.Sinusoidal(amplitude=1.5, frequency=0.25)
#Sinusoidal Signal with Amplitude=1 & Frequency=0.5
signal_2 = ts.signals.Sinusoidal(amplitude=1, frequency=0.5)
#Generating the time series
samples_1, regular_time_samples, signals_1, errors_1 = generate_
timeseries(signal=signal_1)
samples_2, regular_time_samples, signals_2, errors_2 = generate_
timeseries(signal=signal_2)
plot_time_series(regular_time_samples,
                 [samples_1, samples_2],
                 "Sinusoidal Waves",
                 legends=["Amplitude = 1.5 | Frequency = 0.25", "Amplitude = 1
| Frequency = 0.5"])
```

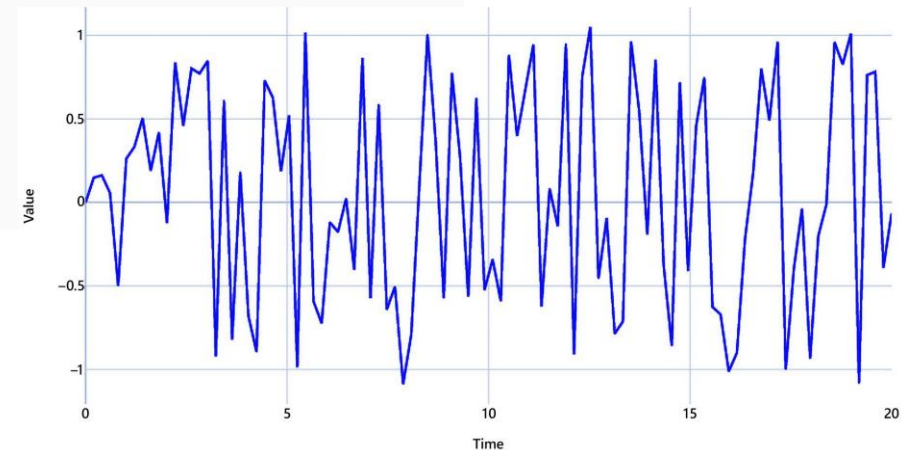


Cyclical or Seasonal Signals

TimeSynth also has another signal called PseudoPeriodic.

This is like the Sinusoidal class, but the **frequency** and **amplitude** have some **stochasticity**.

```
# PseudoPeriodic signal with Amplitude=1 & Frequency=0.25
signal = ts.signals.PseudoPeriodic(amplitude=1, frequency=0.25)
#Generating Timeseries
samples, regular_time_samples, signals, errors = generate_
timeseries(signal=signal)
plot_time_series(regular_time_samples,
                 samples,
                 "Pseudo Periodic")
```



Autoregressive Signals

Another very **popular** signal in the real world is an **autoregressive (AR) signal**.

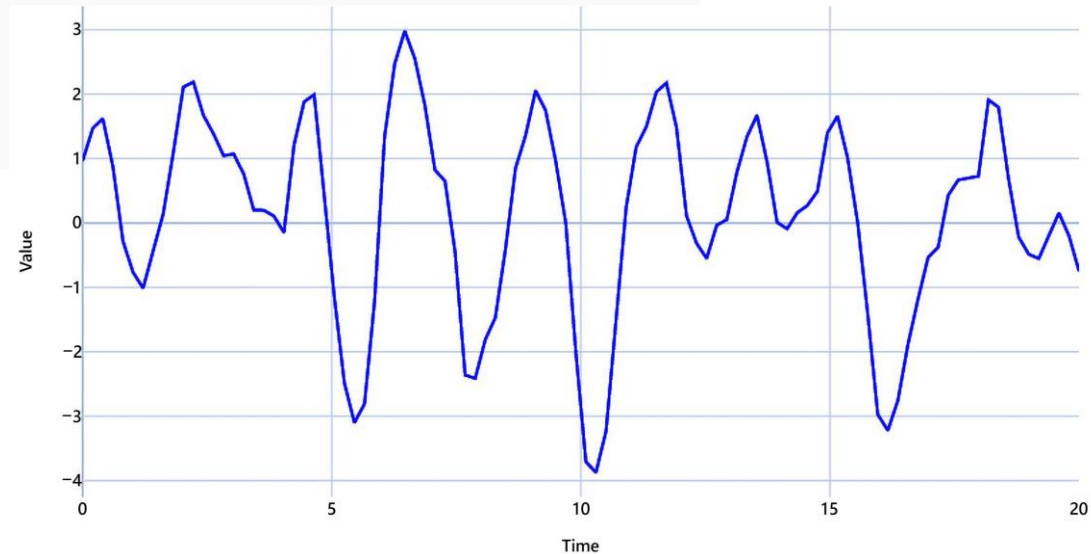
Which refers to when the **value** of a time series for the **current** timestep is **dependent** on the values of the time series in the **previous** timesteps.

This **serial correlation** is a key property of the AR signal, and it is **parametrized** by:

- **Order of serial correlation**, which is the number of previous timesteps the signal is dependent on.
- **Coefficients** to combine the previous timesteps.

Autoregressive Signals

```
# We have re-implemented the class in src because of a bug in TimeSynth  
from src.synthetic_ts.autoregressive import Autoregressive  
# Autoregressive signal with parameters 1.5 and -0.75  
#  $y(t) = 1.5*y(t-1) - 0.75*y(t-2)$   
signal= Autoregressive(ar_param=[1.5, -0.75])  
#Generate Timeseries  
samples, regular_time_samples, signals, errors = generate_  
timeseries(signal=signal)  
plot_time_series(regular_time_samples,  
                 samples,  
                 "Auto Regressive")
```



Mix & Match

#Generating Pseudo Periodic Signal

```
pseudo_samples, regular_time_samples, _, _ = generate_timeseries(signal=ts.  
signals.PseudoPeriodic(amplitude=1, frequency=0.25), noise=ts.noise.  
GaussianNoise(std=0.3))
```

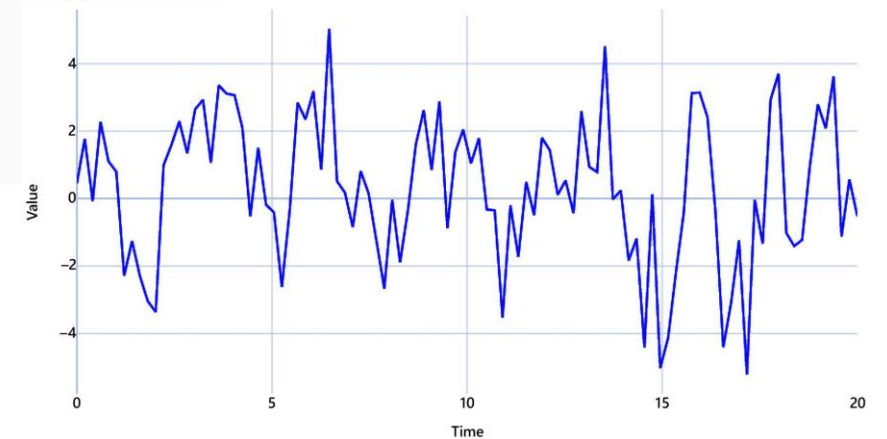
Generating an Autoregressive Signal

```
ar_samples, regular_time_samples, _, _ = generate_timeseries(signal=  
AutoRegressive(ar_param=[1.5, -0.75]))
```

Combining the two signals using a mathematical equation

```
ts = pseudo_samples*2+ar_samples
```

```
plot_time_series(regular_time_samples,  
                 ts,  
                 "Pseudo Periodic with AutoRegression and White Noise")
```



Stationary and non-stationary TS

There are multiple ways to look at **stationarity**, but one of the clearest and most intuitive ways is to think of the **probability distribution** or the data distribution of a time series.

We call a time series stationary when the **probability distribution remains the same at every point in time**. In other words, if you pick different windows in time, the data distribution across all those windows should be the same.

A standard Gaussian distribution is defined by two parameters: the **mean** and the **variance**.

So, there are two ways the **stationarity** assumption can be **broken**, as outlined here:

- **Change in mean** over time
- **Change in variance** over time

Change in mean over time

This is the most popular way a non-stationary time series presents itself.

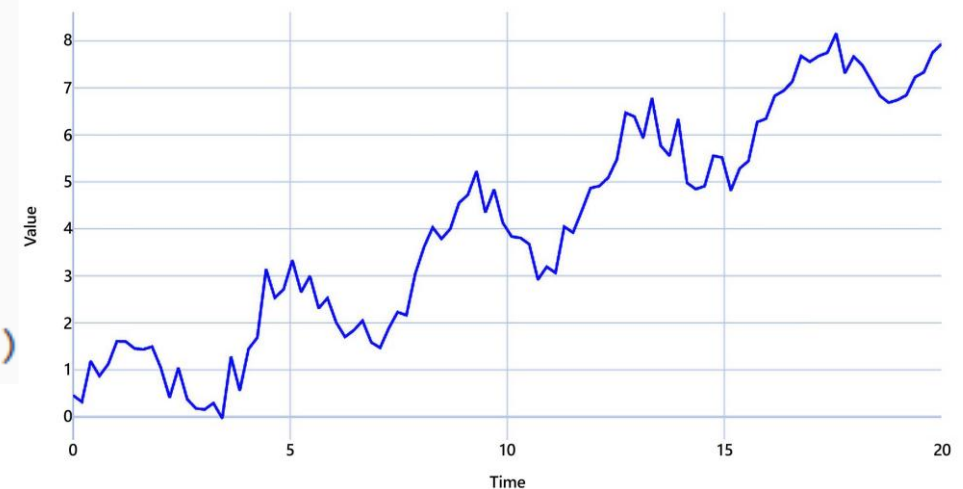
If there is an **upward/downward trend** in the time series, the **mean** across two **windows** of time would **not be the same**.

Another way non-stationarity manifests itself is in the form of **seasonality**.

Suppose we are looking at the time series of **average temperature** measurements **per month** for the last 5 years. From our experience, we know that **temperature peaks during summer and falls in winter**. So, when we take the mean temperature of winter and the mean temperature of summer, they will be different.

Change in mean over time

```
# Sinusoidal Signal with Amplitude=1 & Frequency=0.25
signal=ts.signals.Sinusoidal(amplitude=1, frequency=0.25)
# White Noise with standard deviation = 0.3
noise=ts.noise.GaussianNoise(std=0.3)
# Generate the time series
sinusoidal_samples, regular_time_samples, _, _ = generate_
timeseries(signal=signal, noise=noise)
# Regular_time_samples is a linear increasing time axis and can be used as a
trend
trend = regular_time_samples*0.4
# Combining the signal and trend
ts = sinusoidal_samples+trend
plot_time_series(regular_time_samples,
                 ts,
                 "Sinusoidal with Trend and White Noise")
```



Change in variance over time

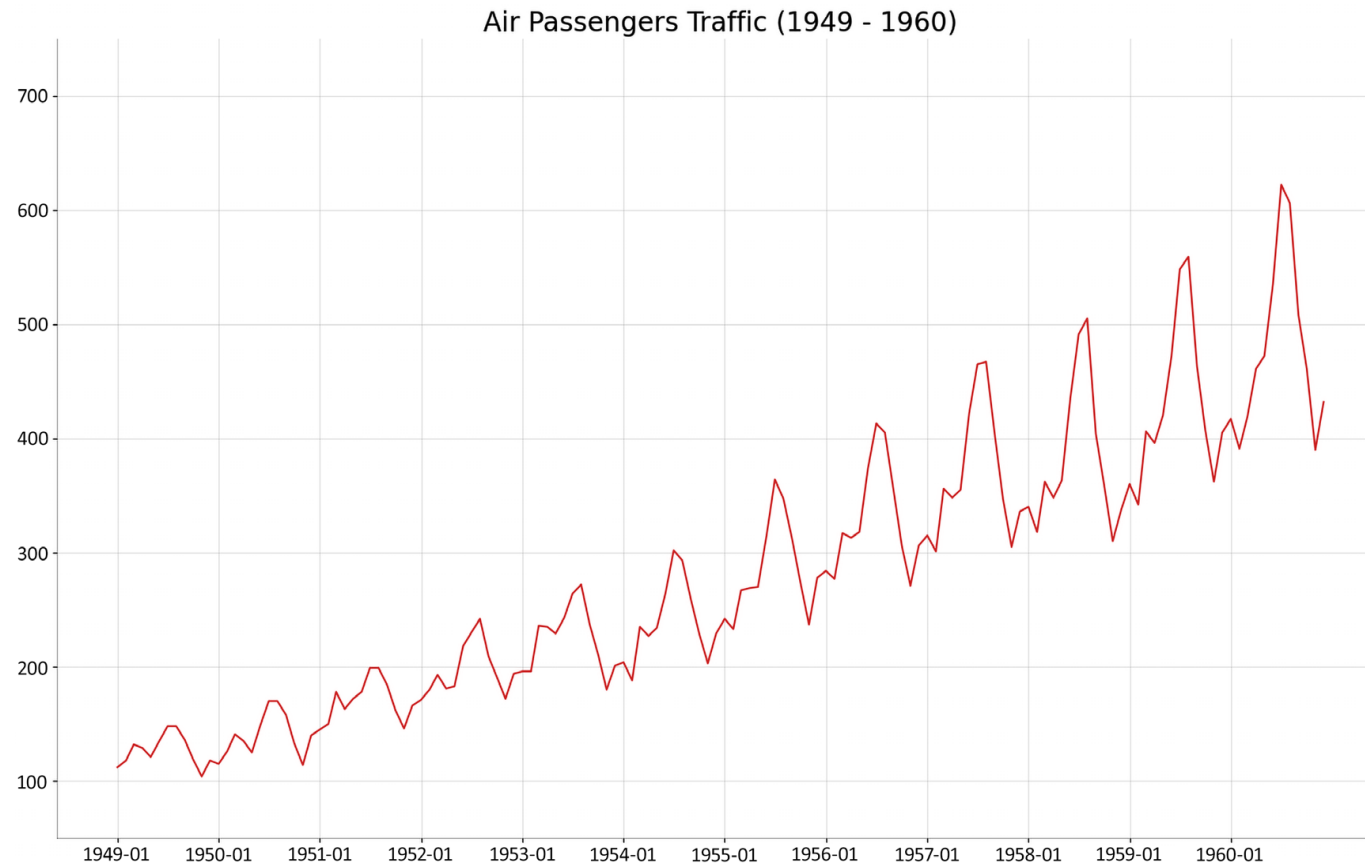
Non-stationarity can also present itself in the **fluctuating variance** of a time series.

If the time series **starts** off with **low variance** and as time progresses, the **variance keeps getting bigger** and bigger, we have a non-stationary time series.

In statistics, it's called heteroscedasticity.

Change in variance over time

The **Air Passengers** dataset is a classic example of a **heteroscedastic** time series.



What can we forecast?

The most basic assumption when we forecast a time series is that **the future depends on the past.**

But not all time series are equally predictable.

Try to rank these in order of predictability (from easiest to hardest):

- High tide next Monday
- Lottery numbers next Sunday
- The stock price of Tesla next Friday

What can we forecast?

We can **forecast stock** prices with the advanced techniques covered in the course and **get rich!**



Share prices are not a function of their past values but an **anticipation** of their **future** values.

Financial stock prices typically have a very **low signal-to-noise ratio**.



The efficient market hypothesis (**EMH**) states that stock prices already reflect all publicly known information, so if a price change can be predicted, that expectation is usually already built into the market price. But it's **not true**.

What can we forecast?

Three main factors for predictability:

- **Understanding the DGP:**

The better you understand the DGP, the higher the predictability of a time series.

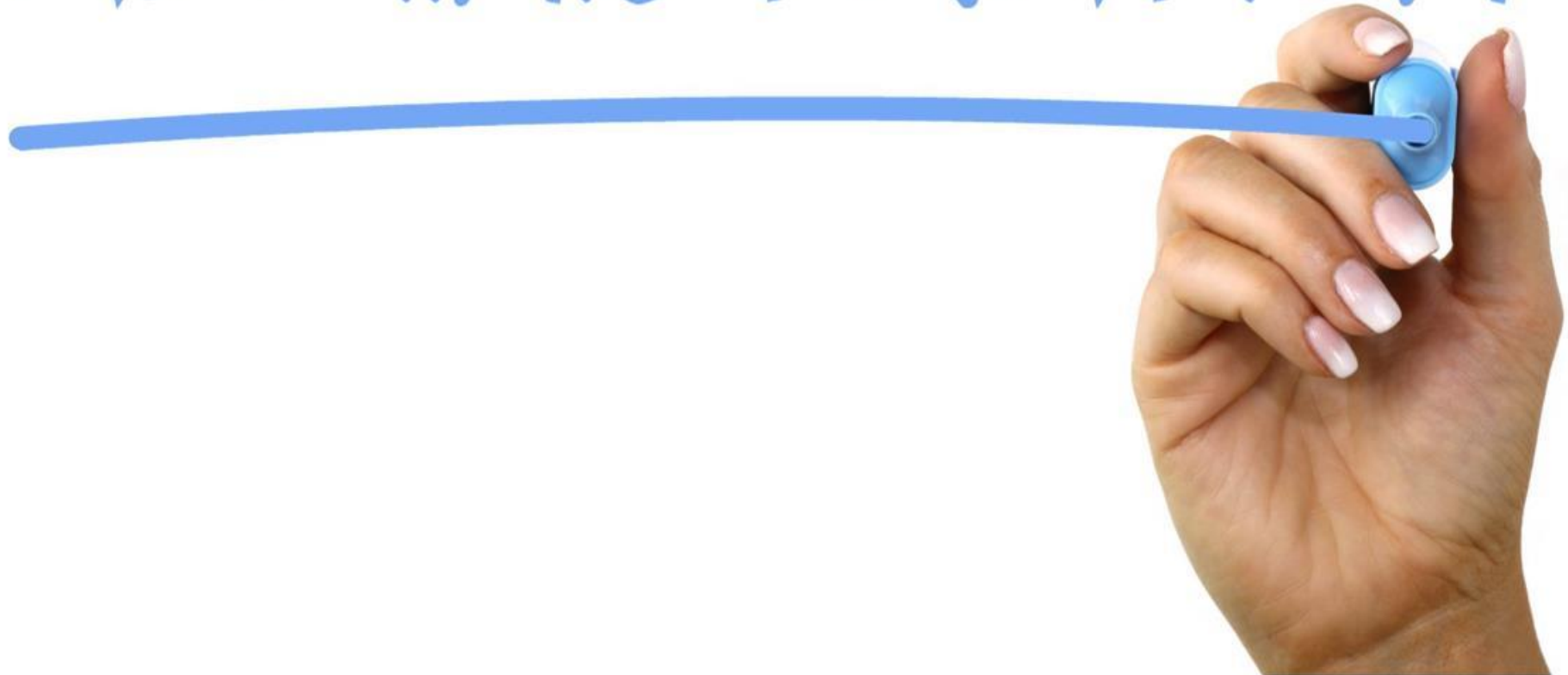
- **Amount of data:**

The more data you have, the better your predictability is.

- **Adequately repeating pattern:**

For any mathematical model to work well, there should be an adequately repeating pattern in your time series. The more repeatable the pattern is, the better your predictability is.

ADMINISTRATION



Assessments

4 homework assignments (40%)

purely practical,
Submitted individually.

Final Project (50%)

end-to-end analytics & forecasting on a chosen dataset
written a report,
code,
and short presentation.

Participation (10%)

~~attendance in class (at least 80%),~~
code reviews,
discussion,
and peer feedback.



Course Content

- 1) Introduction and the Python time-series stack
- 2) EDA and decomposition (trend/seasonality, stationarity)
- 3) Baselines and exponential smoothing (ETS/state-space)
- 4) ARIMA family (ARIMA/SARIMA/SARIMAX)
- 5) Backtesting and evaluation metrics
- 6) Feature engineering and ML forecasters
- 7) Unsupervised patterns and anomaly/change-point detection
- 8) Probabilistic forecasting (quantiles, Prophet/BSTS)
- 9) Deep learning I: RNN/GRU/LSTM
- 10) Deep learning II: CNN/TCN, N-BEATS, Transformers
- 11) Multivariate time series and causality (VAR/VECM, Granger)
- 12) Online/streaming learning and drift detection
- 13) Special topics and deployment (hierarchical/grouped, intermittent demand, packaging/APIs)



Course Schedule

1 – ~~Wed. 11.3 at 17:00-20:00 via Zoom~~

2 – Wed. 18.3 at 17:00-20:00 via Zoom

3 – Wed. 25.3 at 17:00-20:00 via Zoom

4 – Wed. 15.4 at 17:00-20:00 via Zoom

5 – Wed. 29.4 at 17:00-20:00 via Zoom

6 & 7 – **Thu. 7.5 at 13:00-18:00 at BGU campus**

8 – Wed. 20.5 at 17:00-20:00 via Zoom

9 & 10 – **Thu. 28.5 at 13:00-18:00 at BGU campus**

11 & 12 – **Thu. 11.6 at 13:00-18:00 at BGU campus**

13 – Wed. 24.6 at 17:00-20:00 via Zoom

